

eudoroolivares2016 CMRNLP-103: Remove empty resource statement ✓ bb0ac5d · 2 months ago		
.continue/rules	CMRNLP-101: Implement get_...	3 months ago
.github	CMRNLP-103: v1.0 MCP serve...	2 months ago
docs	CMRNLP-103: v1.0 MCP serve...	2 months ago
middleware	Adding langfuse and tests to t...	7 months ago
migrations	CMRNLP-103: v1.0 MCP serve...	2 months ago
models	CMRNLP-103: v1.0 MCP serve...	2 months ago
prompts	CMRNLP-103: v1.0 MCP serve...	2 months ago
scripts	CMRNLP-103: v1.0 MCP serve...	2 months ago
terraform	CMRNLP-103: Remove empty ...	2 months ago
tests	CMRNLP-103: v1.0 MCP serve...	2 months ago
tools	CMRNLP-103: v1.0 MCP serve...	2 months ago
util	CMRNLP-103: v1.0 MCP serve...	2 months ago
.coderabbit.yaml	CMRNLP-101: Fixes deployme...	3 months ago
.copilotignore	CMRNLP-80: Consolidate dis...	6 months ago
.gitignore	CMRNLP-105: Implement get...	2 months ago
.pre-commit-config.yaml	CMRNLP-103: v1.0 MCP serve...	2 months ago
McpServerDockerfile	CMRNLP-101: Implement get_...	3 months ago
McpServerDockerfile.d...	CMRNLP-101: Fixes deployme...	3 months ago
README.md	CMRNLP-103: v1.0 MCP serve...	2 months ago

 loader.py	CMRNLP-103: v1.0 MCP serve...	2 months ago
 pyproject.toml	CMRNLP-103: v1.0 MCP serve...	2 months ago
 server.py	CMRNLP-101: Implement get_...	3 months ago
 uv.lock	CMRNLP-103: v1.0 MCP serve...	2 months ago

README



earthdata-mcp

An MCP (Model Context Protocol) server providing LLM agents with direct access to NASA's Common Metadata Repository (CMR). This integration enables users to agentically discover, verify, and access Earth science datasets through natural language interfaces.

To start querying Earthdata immediately, see [Connecting to the Server](#). For a detailed breakdown of how tool inputs and outputs map to the underlying CMR APIs and UMM schemas, see the [Parameter Support Reference](#).

Available Tools

- **get_keywords** : Discovers official Earthdata scientific vocabulary terms (from NASA KMS) to translate colloquial user inputs (e.g. "rain") into precise search labels (e.g. "PRECIPITATION AMOUNT").
- **get_collections** : Searches for datasets (collections) using scientific keywords, instruments, platforms, or spatial/temporal constraints.
- **get_granules** : Searches for specific data files (granules) within a collection. Used to verify actual data availability for a given time and location.
- **get_services** : Discovers data access endpoints (OPeNDAP, Harmony) and visualization layers (WMS/WMTS) associated with a collection.
- **get_tools** : Finds web portals (e.g., Giovanni, Worldview) and downloadable software (e.g., Panoply) associated with a collection, returning URLs and deep-linking templates.
- **get_citations** : Discovers citation records (publications, DOIs) associated with a collection, or looks up citations directly by identifier.
- **get_variables** : Discovers scientific variables and measurements associated with a collection, or looks up variables by keyword. Use this to understand specific data parameters (scale, offset, fill values) before downloading or analyzing data.

Agent Workflow Instructions

The server provides system instructions (`prompts/instructions.py`) that enforce a **Discover** → **Verify** → **Access** workflow for LLM clients:

1. **Discover:** Find relevant collections using `get_collections` .
2. **Verify:** Use `get_granules` to confirm data actually exists for the user's requested region/time, as collections often declare global coverage regardless of gaps.
3. **Access:** Instruct users to use the `earthaccess` Python library for authentication and downloading, providing relevant code snippets.

Project Structure

The repository is structured around a few core domains:

- `server.py` & `loader.py` : The FastMCP server entry point and dynamic tool registration logic.
- `prompts/` : System prompts and instructions that define the LLM's workflow and persona.
- `tools/` : Self-contained MCP tools wrapping NASA CMR APIs (`get_collections` , `get_granules` , `get_keywords` , `get_services` , `get_tools` , `get_citations` , `get_variables`).
- `models/` : Pydantic models for tool input validation and standardized CMR API responses.
- `tests/` : Comprehensive test suite (using `pytest`) covering server initialization, tool logic, and mocked CMR API responses.
- `docs/` : Project documentation separated into `consumers/` and `developers/` .

Note on Legacy Code: The ingestion and embedding pipelines (including the `discover_data` tool, `lambdas/` directory, and associated infrastructure) are currently being deprecated. The architecture is transitioning to rely purely on direct, real-time CMR API integrations.

For Consumers: Connecting to the Server

The Earthdata MCP server is deployed remotely and communicates via the official Model Context Protocol **Streamable HTTP** transport. To use the server, configure your MCP-compatible client to point to our endpoint.

Connection URL

Configure your client to connect to the following HTTP endpoint. Most MCP clients require these standard parameters:

- **Transport Type:** `streamable-http`
- **URL:** `https://cmr.earthdata.nasa.gov/mcp/v1`

- **Timeout:** `60000` (Recommended: 60 seconds to allow for complex spatial/temporal queries)

Works with:

- Claude Code CLI
- VS Code MCP extensions
- LibreChat
- Any MCP-compatible client that supports Streamable HTTP transport

For Developers: Local Environment

If you want to contribute to the server or run it locally, follow these steps.

Prerequisites

- Python 3.13+
- [uv](#) package manager

Installation

```
# Clone and enter directory
git clone <repo-url>
cd earthdata-mcp

# Install dependencies
uv sync

# Install dev dependencies (for testing)
uv sync --extra dev
```



Starting the Local Server

We recommend running the server in HTTP mode for local development and testing:

```
uv run server.py http
```



The server will start and be available at `http://127.0.0.1:5001/mcp/v1`.

Development & Testing

See [docs/developers/](#) for developer guides:

- [Adding a New Tool](#)
- [Adding an Environment Variable](#)
- [Troubleshooting Deployments](#) (Debugging 503 errors, AWS ECS crash loops, and local startup issues)

Running Tests

```
# Run all tests
uv run pytest

# Run with verbose output
uv run pytest -v

# Run specific test file
uv run pytest tests/test_server.py
```



MCP Inspector (Interactive Testing)

1. Start the server:

```
uv run server.py http
```



2. Launch inspector:

```
npx @modelcontextprotocol/inspector
```



3. Connect at `http://localhost:6274` :

- Transport Type: **Streamable HTTP**
- URL: `http://localhost:5001/mcp/v1`

Deployment

The application deploys to AWS via Bamboo CI/CD.

Releases

 4 tags

Packages

No packages published

Contributors 7



Languages

